

Mudcard

- **kfold, I still don't understand how it works and I cannot see the picture in the Jupyter notebook.**
 - If you don't see the picture, it likely means that your notebook was moved or you didn't download the whole github repo
 - If you want to run the code, it's not enough to grad the notebook, you need to download the data and figures folders as well
 - If you click on the markdown cell where the kfold picture should show up, you'll this line:
 - `<center></center>`
 - So the pic is located one level up in the directory structure (`../`) and in the figures folder
 - You can read more about Kfold [here](#)
- **How does the shuffling in KFold work during the training of the ML model?**
 - The data is split before you train the model, not during model training
 - You can read more about Kfold [here](#)
- **All good, I just keep getting surprised with how often my mud cards show up; do people just not answer?**
 - Yeah, not everyone fills out the mud card
- **Can we review what the probability distribution function D is for IID?**
 - We do not know what the distribution D is, it can be anything.
 - What we assume about D is that it doesn't change as we draw samples from it (identically distributed), and the feature values of the next sample are not determined by any of the previous samples (independent)
- **Can we review the ""Why shuffling IID data is important?"" section or the final graph in this lecture, please?**
 - it's important if your data happens to be stored in an ordered way
 - for example, you have 100 points and you want to create 5 folds
 - if the data is ordered, the first 20 rows could be in class 1, the points in rows 21-40 could also be in class 1, points in rows 41-60 are in class 0, etc.
 - eventhough the data is IID, the points are ordered, so when you create the 5 folds, fold 1 only contain class 1 point, fold 2 only contains class 1 points, fold 3 only contains class 0 points, etc.
 - This is not good because you'd want roughly the same proportion of class 1 and 0 points in each fold
 - If you shuffle the dataset before the folds are created, this will be guaranteed.
- **What is the difference between test sets and validation sets?**
 - The validation set is used to select the best ML model
 - The test set is used to measure how well that best model performs on previously not seen points when the model is deployed
- **For the last choice in quiz 1, why would it be IID if there is only 1 patient? Wouldn't that still be a time dependet process?**
 - it wouldn't necessarily be iid, it depends on the problem you try to solve
 - it's important to internalize that one dataset can be used to answer a bunch of different ML questions and the splitting strategy depends on the ML question
- **How to decide splitting strategy in real ML application?**
 - If your dataset is iid, you can use either the simple train/val/test split or kfold
 - kfold is sometimes preferred if your dataset is small
 - We will cover how to split your data if your dataset is not iid, that's when things get more complicated
- **What's the funcation and importance of validation part?**
 - it is used to select which ML model is the best
- **Question about the iid assumption (papaya example): My unit of interest is the individual papaya, not the tree. Papayas from the same tree can still differ in sweetness. Suppose an island has only 5 papaya trees, each producing 20 papayas, and I want to label each papaya as tasty or not. 1. If I go tree by tree and measure each papaya in order, is the data on these 100 papayas iid? 2. If I pick all 100 papayas first and then measure them one by one (e.g., in random order), is it iid? 3. Since the same tree can produce papayas of different sweetness, is coming from the same tree enough on its own to break independence? 4. If the island were expanded to the whole Earth, would the answers to the same questions (measuring tree by tree vs. picking everything and measuring one by one) change? 5. If so, at which tree does the condition change? Under what conditions can the data be treated as approximately iid: the number of trees, how many papayas are measured per tree, or something else?**
 - Nice! I like this line of questioning!

- You are right, just because papayas from the same tree are different from each other does not necessarily mean that the papayas are independent!
- For example papayas from tree i could have a high probability of sweetness (let's call this p_i) because of the soil properties, amount of sun tree i receives, etc.
- Papayas from a different tree j could have a different probability of sweetness p_j
- So if you know which tree the papaya came from, the papaya samples are likely not independent unless each tree has the exact same probability of sweetness.
- In my example, you could say for example that I sample the papayas in a market and I do not know which tree each papaya came from. Then I can assume the papayas are iid
- For point 5, there is no magical point where increasing the population makes the observations iid. If you know which tree a papaya came from, the problem is not iid. If you do not have that info, you can assume the papayas are iid.
- **How do you guarantee the split is not biased? Even that `train_test_split()` is black box**
 - It shouldn't be.
 - You can check the source code [here](#), it is less than 40 lines of code
 - You should understand these algorithms well enough to be able to implement them yourself.
 - In fact, it would be considered a relatively easy coding interview question to implement k-fold splitting from scratch using only python and numpy or pandas/polars.
 - Alumni of the data science master's program reported much tougher coding interview questions recently. Like "Implement support vector machines from scratch using python and numpy only." This is beyond DATA1030 but covered in DATA2060.

Lecture 6: Data preprocessing

By the end of this lecture, you will be able to

- apply one-hot encoding on categorical features
- apply ordinal encoding on ordinal features
- apply scaling and normalization to continuous variables

The supervised ML pipeline

0. Data collection/manipulation: you might have multiple data sources and/or you might have more data than you need

- you need to be able to read in datasets from various sources (like csv, excel, SQL, parquet, etc)
- you need to be able to filter the columns/rows you need for your ML model
- you need to be able to combine the datasets into one dataframe

1. Exploratory Data Analysis (EDA): you need to understand your data and verify that it doesn't contain errors

- do as much EDA as you can!

2. Split the data into different sets: most often the sets are train, validation, and test (or holdout)

- practitioners often make errors in this step!
- you can split the data randomly, based on groups, based on time, or any other non-standard way if necessary to answer your ML question

3. Preprocess the data: ML models only work if X and Y are numbers! Some ML models additionally require each feature to have 0 mean and 1 standard deviation (standardized features)

- often the original features you get contain strings (for example a gender feature would contain 'male', 'female', 'non-binary', 'unknown') which needs to be transformed into numbers
- often the features are not standardized (e.g., age is between 0 and 100) but it needs to be standardized

4. Choose an evaluation metric: depends on the priorities of the stakeholders

- often requires quite a bit of thinking and ethical considerations

5. Choose one or more ML techniques: it is highly recommended that you try multiple models

- start with simple models like linear or logistic regression
- try also more complex models like nearest neighbors, support vector machines, random forest, etc.

6. Tune the hyperparameters of your ML models (aka cross-validation or hyperparameter tuning)

- ML techniques have hyperparameters that you need to optimize to achieve best performance
- for each ML model, decide which parameters to tune and what values to try
- loop through each parameter combination
 - train one model for each parameter combination
 - evaluate how well the model performs on the validation set
- take the parameter combo that gives the best validation score
- evaluate that model on the test set to report how well the model is expected to perform on previously unseen data

7. Interpret your model: black boxes are often not useful

- check if your model uses features that make sense (excellent tool for debugging)
- often model predictions are not enough, you need to be able to explain how the model arrived to a particular prediction (e.g., in health care)

Problem description, why preprocessing is necessary

Data format suitable for ML: 2D numerical values.

X	feature_1	feature_2	...	feature_j	...	feature_m	y
data_point_1	x_11	x_12	...	x_1j	...	x_1m	y_1
data_point_2	x_21	x_22	...	x_2j	...	x_2m	y_2
...	...	...	...	...	...	...	...
data_point_i	x_i1	x_i2	...	x_ij	...	x_im	y_i
...	...	...	...	...	...	...	...
data_point_n	x_n1	x_n2	...	x_nj	...	x_nm	y_n

Data almost never comes in a format that's directly usable in ML.

- let's check the adult data

```
In [ ]: import pandas as pd
        from sklearn.model_selection import train_test_split

        df = pd.read_csv('../data/adult_data.csv')

        # let's separate the feature matrix X, and target variable y
        y = df['gross-income'] # remember, we want to predict who earns more than 50k or less than 50k
        X = df.loc[:, df.columns != 'gross-income'] # all other columns are features

        random_state = 42

        # first split to separate out the training set
        X_train, X_other, y_train, y_other = train_test_split(X,y,train_size = 0.6,random_state=random_state)

        # second split to separate out the validation and test sets
        X_val, X_test, y_val, y_test = train_test_split(X_other,y_other,train_size = 0.5,random_state=random_state)

        print('training set')
        print(X_train.head()) # lots of strings!
        print(y_train.head()) # even our labels are strings and not numbers!
```

scikit-learn transformers to the rescue!

Preprocessing is done with various transformers. All transformers have three methods:

- **fit** method: estimates parameters necessary to do the transformation,

- **transform** method: transforms the data based on the estimated parameters,
- **fit_transform** method: both steps are performed at once, this can be faster than doing the steps separately.

Transformers we cover today

- **OneHotEncoder** - converts categorical features into dummy arrays
- **OrdinalEncoder** - converts ordinal features into an integer array
- **MinMaxScaler** - scales continuous variables to be between 0 and 1
- **StandardScaler** - standardizes continuous features by removing the mean and scaling to unit variance

By the end of this lecture, you will be able to

- apply one-hot encoding on categorical features
- apply ordinal encoding on ordinal features
- apply scaling and normalization to continuous variables

Unordered categorical data: one-hot encoder

- some categories cannot be ordered. e.g., workclass, relationship status

```
In [ ]: from sklearn.preprocessing import OneHotEncoder
        help(OneHotEncoder)
```

```
In [ ]: # toy example
train = {'gender':['Male','Female','Unknown','Male','Female','Female'],\
        'browser':['Safari','Safari','Internet Explorer','Chrome','Chrome','Internet Explorer']}
test = {'gender':['Female','Male','Unknown','Female'],'browser':['Chrome','Firefox','Internet Explorer'],'S

Xtoy_train = pd.DataFrame(train)
Xtoy_test = pd.DataFrame(test)

ftrs = ['gender','browser']

# initialize the encoder
enc = OneHotEncoder(sparse_output=False, handle_unknown='ignore') # handle_unknown='ignore' # by default,
# fit the training data
enc.fit(Xtoy_train)
print('categories:',enc.categories_)
print('feature names:',enc.get_feature_names_out(ftrs))
# transform X_train
X_train_ohe = enc.transform(Xtoy_train)
#print(X_train_ohe)
# do all of this in one step
X_train_ohe = enc.fit_transform(Xtoy_train)
print('X_train transformed')
print(X_train_ohe)

# transform X_test
X_test_ohe = enc.transform(Xtoy_test)
print('X_test transformed')
print(X_test_ohe)
```

```
In [ ]: # apply OHE to the adult dataset

# let's collect all categorical features first
onehot_ftrs = ['workclass','marital-status','occupation','relationship','race','sex','native-country']
# initialize the encoder
enc = OneHotEncoder(sparse_output=False,handle_unknown='ignore') # by default, OneHotEncoder returns a spa
# fit the training data
enc.fit(X_train[onehot_ftrs])
print('feature names:',enc.get_feature_names_out(onehot_ftrs))
print(len(enc.get_feature_names_out(onehot_ftrs)))
```

```
In [ ]: # transform X_train
onehot_train = enc.transform(X_train[onehot_ftrs])
print('transformed train features:')
```

```
print(onehot_train)
# transform X_val
onehot_val = enc.transform(X_val[onehot_ftrs])
print('transformed val features:')
print(onehot_val)
# transform X_test
onehot_test = enc.transform(X_test[onehot_ftrs])
print('transformed test features:')
print(onehot_test)
```

By the end of this lecture, you will be able to

- apply one-hot encoding on categorical features
- **apply ordinal encoding on ordinal features**
- apply scaling and normalization to continuous variables

Ordered categorical data: OrdinalEncoder

- use it on categorical features if the categories can be ranked or ordered
 - educational level in the adult dataset
 - reaction to medication is described by words like 'severe', 'no response', 'excellent'
 - any time you know that the categories can be clearly ranked

```
In [ ]: from sklearn.preprocessing import OrdinalEncoder
        help(OrdinalEncoder)
```

```
In [ ]: # toy example
import pandas as pd

train_edu = {'educational level': ['Bachelors', 'Masters', 'Bachelors', 'Doctorate', 'HS-grad', 'Masters']}
test_edu = {'educational level': ['HS-grad', 'Masters', 'Masters', 'College', 'Bachelors']}

Xtoy_train = pd.DataFrame(train_edu)
Xtoy_test = pd.DataFrame(test_edu)

# initialize the encoder
cats = [['HS-grad', 'College', 'Bachelors', 'Masters', 'Doctorate']]

enc = OrdinalEncoder(categories = cats) # The ordered list of
# categories need to be provided. By default, the categories are alphabetically ordered!

# fit the training data
enc.fit(Xtoy_train)
# print the categories - not really important because we manually gave the ordered list of categories
print(enc.categories_)
# transform X_train. We could have used enc.fit_transform(X_train) to combine fit and transform
X_train_oe = enc.transform(Xtoy_train)
print(X_train_oe)
# transform X_test
X_test_oe = enc.transform(Xtoy_test) # OrdinalEncoder always throws an error message if
# it encounters an unknown category in test
print(X_test_oe)
```

```
In [ ]: # apply OE to the adult dataset
# initialize the encoder
ordinal_ftrs = ['education'] # if you have more than one ordinal feature, add the feature names here
ordinal_cats = [['Preschool', '1st-4th', '5th-6th', '7th-8th', '9th', '10th', '11th', '12th', 'HS-grad', 'Some-college', 'Assoc-voc', 'Assoc-acdm', 'Bachelors', 'Masters', 'Prof-school', 'Doctorate']]
# ordinal_cats must contain one list per ordinal feature! each list contains the ordered list of categories
# of the corresponding feature

enc = OrdinalEncoder(categories = ordinal_cats) # By default, the categories are alphabetically ordered
# which is NOT what you want usually.

# fit the training data
enc.fit(X_train[ordinal_ftrs]) # the encoder expects a 2D array, that's why the column name is in a list

# transform X_train. We could use enc.fit_transform(X_train) to combine fit and transform
ordinal_train = enc.transform(X_train[ordinal_ftrs])
```

```
print('transformed train features:')
print(ordinal_train)
# transform X_val
ordinal_val = enc.transform(X_val[ordinal_ftrs])
print('transformed validation features:')
print(ordinal_val)
# transform X_test
ordinal_test = enc.transform(X_test[ordinal_ftrs])
print('transformed test features:')
print(ordinal_test)
```

Quiz 1

Please explain how you would encode the race feature below and what would be the output of the encoder. Do not write code. The goal of this quiz is to test your conceptual understanding so write text and the output array.

race = ['Amer-Indian-Eskimo', 'White', 'Black', 'Asian-Pac-Islander', 'Black', 'White', 'White']

By the end of this lecture, you will be able to

- apply one-hot encoding on categorical features
- apply ordinal encoding on ordinal features
- **apply scaling and normalization to continuous variables**

Continuous features: MinMaxScaler

- If the continuous feature values are reasonably bounded, MinMaxScaler is a good way to scale the features.
- Age is expected to be within the range of 0 and 100.
- Number of hours worked per week is in the range of 0 to 80.
- If unsure, plot the histogram of the feature to verify or just go with the standard scaler!

```
In [ ]: from sklearn.preprocessing import MinMaxScaler
        help(MinMaxScaler)
```

```
In [ ]: # toy data
# let's assume we have two continuous features:
train = {'age': [32, 65, 13, 68, 42, 75, 32], 'number of hours worked': [0, 40, 10, 60, 40, 20, 40]}
test = {'age': [83, 26, 10, 60], 'number of hours worked': [0, 40, 0, 60]}

# (value - min) / (max - min), if value is 32, min is 13 and max is 75, then we have 19 / 62 = 0.3064

Xtoy_train = pd.DataFrame(train)
Xtoy_test = pd.DataFrame(test)

scaler = MinMaxScaler()
scaler.fit(Xtoy_train)
print(scaler.transform(Xtoy_train))
print(scaler.transform(Xtoy_test)) # note how scaled X_test contains values larger than 1 and smaller than
```

```
In [ ]: # adult data

minmax_ftrs = ['age', 'hours-per-week']

scaler = MinMaxScaler()
scaler.fit(X_train[minmax_ftrs])
print(scaler.transform(X_train[minmax_ftrs]))
print(scaler.transform(X_val[minmax_ftrs]))
print(scaler.transform(X_test[minmax_ftrs]))
```

Continuous features: StandardScaler

- If the continuous feature values follow a tailed distribution, StandardScaler is better to use!
- Salaries are a good example. Most people earn less than 100k but there are a small number of super-rich people.

```
In [ ]: from sklearn.preprocessing import StandardScaler
        help(StandardScaler)

In [ ]: # toy data
        train = {'salary': [50_000, 75_000, 40_000, 1_000_000, 30_000, 250_000, 35_000, 45_000]}
        test = {'salary': [25_000, 55_000, 1_500_000, 60_000]}

        Xtoy_train = pd.DataFrame(train)
        Xtoy_test = pd.DataFrame(test)

        scaler = StandardScaler()
        print(scaler.fit_transform(Xtoy_train))
        print(scaler.transform(Xtoy_test))

In [ ]: # adult data

        std_fts = ['capital-gain', 'capital-loss']
        scaler = StandardScaler()
        print(scaler.fit_transform(X_train[std_fts]))
        print(scaler.transform(X_val[std_fts]))
        print(scaler.transform(X_test[std_fts]))
```

Quiz 2

Which of these features could be safely preprocessed by the minmax scaler?

- number of minutes spent on the website in a day
- number of days a year spent abroad in a year
- USD donated to charity

How and when to do preprocessing in the ML pipeline?

- **APPLY TRANSFORMER.FIT ONLY ON YOUR TRAINING DATA!** Then transform the validation and test sets.
- One of the most common mistake practitioners make is leaking statistics!
 - fit_transform is applied to the whole dataset, then the data is split into train/validation/test
 - this is wrong because the test set statistics impacts how the training and validation sets are transformed
 - but the test set must be separated from train and val, and val must be separated from train
 - or fit_transform is applied to the train, then fit_transform is applied to the validation set, and fit_transform is applied to the test set
 - this is wrong because the relative position of the points change

 No description has been provided for this image

Scikit-learn's pipelines

- The steps in the ML pipeline can be chained together into a scikit-learn pipeline which consists of transformers and one final estimator which is usually your classifier or regression model.
- It neatly combines the preprocessing steps and it helps to avoid leaking statistics.

https://scikit-learn.org/stable/auto_examples/compose/plot_column_transformer_mixed_types.html

```
In [ ]: import pandas as pd
        import numpy as np

        from sklearn.compose import ColumnTransformer
        from sklearn.pipeline import Pipeline
        from sklearn.preprocessing import StandardScaler, OneHotEncoder, OrdinalEncoder, MinMaxScaler
        from sklearn.model_selection import train_test_split

        #np.random.seed(0)

        df = pd.read_csv('../data/adult_data.csv')

        # let's separate the feature matrix X, and target variable y
        y = df['gross-income'] # remember, we want to predict who earns more than 50k or less than 50k
```

```
X = df.loc[:, df.columns != 'gross-income'] # all other columns are features

random_state = 42

# first split to separate out the training set
X_train, X_other, y_train, y_other = train_test_split(X,y,train_size = 0.6,random_state=random_state)

# second split to separate out the validation and test sets
X_val, X_test, y_val, y_test = train_test_split(X_other,y_other,train_size = 0.5,random_state=random_state)
```

```
In [ ]: # collect which encoder to use on each feature
# needs to be done manually
ordinal_ftrs = ['education']
ordinal_cats = [[' Preschool',' 1st-4th',' 5th-6th',' 7th-8th',' 9th',' 10th',' 11th',' 12th',' HS-grad','\
                ' Some-college',' Assoc-voc',' Assoc-acdm',' Bachelors',' Masters',' Prof-school',' Doctorate'],
                ['workclass','marital-status','occupation','relationship','race','sex','native-country']]

onehot_ftrs = ['workclass','marital-status','occupation','relationship','race','sex','native-country']
minmax_ftrs = ['age','hours-per-week']
std_ftrs = ['capital-gain','capital-loss']

# collect all the encoders
preprocessor = ColumnTransformer(
    transformers=[
        ('ord', OrdinalEncoder(categories = ordinal_cats), ordinal_ftrs),
        ('onehot', OneHotEncoder(sparse_output=False,handle_unknown='ignore'), onehot_ftrs),
        ('minmax', MinMaxScaler(), minmax_ftrs),
        ('std', StandardScaler(), std_ftrs)])

clf = Pipeline(steps=[('preprocessor', preprocessor)]) # for now we only preprocess
                                                    # later on we will add other steps here

X_train_prep = clf.fit_transform(X_train)
X_val_prep = clf.transform(X_val)
X_test_prep = clf.transform(X_test)

print(X_train.shape)
print(X_train_prep.shape)
print(X_train_prep)
```

Mudcard

```
In [ ]:
```